Part 1:

1. I propose a row key of the following structure: <hashed_vin>#<make>. This would prevent hotspotting as the rows would be well distributed across nodes - as the vins are unique and when hashed would be random. This would make queries harder because rows are sorted lexicographically and various makes would be randomly scattered down the rows.

2. Column families should be grouped by access patterns: so one family could be a vehicle-specific field like vin, make, year and model. Another family could be location specific like address, city and postal code.

3. The pro of using a hashed shard key (on vin, for example) is that it evenly distributes the rows so no single node is overloaded. But the con is that it makes it harder to make queries based on a certain attribute (like make or model) more inefficient. For a range-based shard key, the related data is kept closer together physically which leads to faster scans and queries. However, it could lead to uneven data distribution and more hotspotting.

Part 4:

The average latency for insert-fast was 0.1554 seconds. The average latency for insert-safe was 0.1708 seconds.

The CAP theorem says that distributed systems must trade off between consistency and availability during network partitions. The /insert-fast endpoint prioritizes availability and low latency over strong consistency and durability. Real-world scenarios where this would be acceptable would be a monitoring dashboard or user analytics use case because we would want low latency and faster response times and losing a small amount of data is not critical.